



សាកលវិទ្យាល័យភ្នំពេញអន្តរជាតិ
PHNOM PENH INTERNATIONAL UNIVERSITY

BIG DATA PRESENTATION

Professor: Mr. Min Sothearith

Presenter: Mr. Chang Tong & Ms. Chea Lina

Urban Cart Analytics Term Project

Customer Value, Product Profitability, and Data Trustworthiness

Table of Contents

Table of Contents.....	1
1. Executive Summary.....	3
Top 5 Findings.....	3
Top Recommendations.....	4
2. Introduction & Business Context.....	4
Question 1: Which customers are actually worth investing in?	4
Question 2: Which products are actually profitable?.....	4
Question 3: Can the company's own data be trusted to act on?	5
3. Data Description	5
3.1 Data Sources.....	5
3.2 Database Schema.....	5
3.3 Scale Summary.....	7
3.4 Known Limitations of the Raw Data	7
4. SQL Extraction & Methodology	8
4.1 Query 1: Revenue, Order Count, and AOV by Category	8
4.2 Query 2: Top 20 Customers by Lifetime Spend.....	8
4.3 Query 3: Month-over-Month Revenue Trend (Last 24 Months, LAG() window function)	9
4.4 Query 4: Return Rate by Category	9
4.5 Query 5: Customers Active in All of the Last 3 Calendar Quarters	10
4.6 Query 6: Top 10 Products by Average Rating (≥ 15 reviews)	10
4.7 Query 7: Avg. Session Duration & Pages Viewed by Device (Purchasing Customers Only).....	11
4.8 Query 8: RANK() of Products by Net Revenue Within Category	11
4.9 Query 9: Payment-Method Mix by Country	12
4.10 Query 10: (Own Question) Highest-Margin Category by Country	12
5.1 Legacy Date Standardization.....	13
5.2 Duplicate & Near-Duplicate Customer Records.....	13
5.3 Missing-Value Policy.....	13
5.4 Price Outliers in Products.....	14
5.5 Merging the Supplier Catalog into Products.....	14
5.6 order_items Cleanup: Duplicates and Returns	14
5.7 Reshape: Category $\times$ Month Revenue Matrix	15
5.8 Time-Series Reshape: Weekly Active Customers.....	15
5.9 The Data-Quality Finding That Changed a Conclusion.....	15
6.1 RFM Segmentation.....	16
6.2 Product Similarity / Recommendation.....	17
6.3 Regression via the Normal Equation	17

6.4 Monte Carlo Stockout Simulation.....	18
7.1 Which customer segment generates the most revenue, and what distinguishes it demographically?	19
7.2 Is there seasonality in overall revenue?	20
7.3 Which category has the highest effective margin after discounts and returns?	20
7.4 Do higher review ratings correlate with repeat-purchase behavior?	21
7.5 Which device × country combination shows the highest engagement-to-purchase conversion?	21
7.6 Based on the regression model, what is the projected revenue for the next 2 months?.....	22
7.7 Which 5 products are the strongest stockout risks, and what reorder points are recommended?	23
7.8 What data-quality finding from Phase 2 materially changed a business conclusion?.....	24
Priority 1: Fix revenue reporting immediately.....	25
Priority 2: Reorder “Those Non-Fiction” now.....	25
Priority 3: Build retention specifically around the Champions segment	25
Priority 4: Shift budget toward Q4, away from Q1.....	25
Priority 5: Re-evaluate rating-based loyalty initiatives.....	25
10. Appendices	26
Appendix A: Full SQL Query Listing.....	26
Appendix B: Data Dictionary	28
Appendix C: Additional Charts.....	28
Appendix D: Individual Contribution Statement	29
Appendix E: Reusable Pipeline	29

1. Executive Summary

UrbanCart asked three questions of its data: which customers are actually worth investing in, which products are truly profitable once returns and discounts are accounted for, and whether the company's own data can be trusted enough to act on. This report answers all three, using the company's order, customer, product, review, and web-session data spanning January 2022 through December 2024 — 2,500 customers, 300 products, and roughly 9,000 orders in total.

The single most important finding in this report is not a customer or product insight — it is a data-quality one. When cancelled, returned, and still-pending orders are counted as if they were completed sales, total revenue is overstated by roughly 71% (\$12.97M reported vs. \$7.57M actually earned). Any past decision — revenue targets, inventory commitments, staffing plans — built on raw order counts was very likely built on a number nearly double the truth. This is not a hypothetical risk; it is a mistake that was actually made once during the production of this very report, caught only through a deliberate cross-check (see Section 5.9).

Top 5 Findings

- A small group of customers drives nearly half of all revenue. “Champions,” the top RFM segment, are only about 500 of 2,191 purchasing customers (roughly 23%) but generate \$3.63M — 48% of total revenue. They are not demographically distinct (same average age, same top country as everyone else); they are simply more frequent, more recent, and higher-spending buyers.
- Revenue is seasonal, not flat. November is UrbanCart's strongest month (\$273,696 average); February is consistently the weakest (\$151,670 average) — a swing of roughly 80%. Planning that assumes flat month-to-month demand will misallocate inventory and staff.
- Books & Media is the best category by margin, not by revenue. At a 77.95% effective margin after discounts and returns, it beats every other category by a wide margin — even though Electronics and Beauty & Health bring in more raw revenue. Optimizing for revenue and optimizing for profit point to different “best” categories.
- Review ratings do not predict repeat purchases. The correlation between a customer's average rating and how often they buy is 0.036 — essentially zero. Whatever drives repeat purchases at UrbanCart, it is not reflected in star ratings, and marketing spend built on “improve ratings to drive loyalty” logic is unlikely to pay off.
- One product is on the edge of a near-certain stockout. “Those Non-Fiction” (Books & Media) currently holds 0 units of stock against historical average monthly demand of 1.9 units — a simulated stockout probability of 96.5%. This is an immediate, actionable inventory gap, not a statistical curiosity.

Top Recommendations

- Filter all revenue reporting to completed orders only, immediately. This single change corrects a 71% overstatement and should be treated as a data-governance priority, not a footnote.
- Build retention programs around the Champions segment specifically — they already drive disproportionate revenue and are not defined by any demographic trait leadership could otherwise target.
- Reorder “Those Non-Fiction” immediately; review reorder points for the four other at-risk products identified in Section 7.7.
- Shift Q1 (January–February) marketing and staffing budget toward Q4, in line with the seasonality finding in Section 7.2.
- Re-evaluate any customer-satisfaction initiative that assumes higher ratings drive repeat purchases — the data does not support that assumption at UrbanCart.

2. Introduction & Business Context

UrbanCart is a mid-sized e-commerce retailer selling across six categories — Apparel, Beauty & Health, Books & Media, Electronics, Home & Kitchen, and Sports & Outdoors — to customers in nine countries. Like most growing retailers, UrbanCart's data lives in more than one place: a transactional database that tracks day-to-day orders, an older CRM export inherited from a previous system, and a supplier-provided product catalog that doesn't perfectly line up with the internal product table. Before any of the three business questions below could be answered honestly, this report first had to reconcile those three sources into one trustworthy picture — a project in itself, covered in Section 5.

This analysis was commissioned to answer three questions from leadership:

Question 1: Which customers are actually worth investing in?

UrbanCart has 2,500 customers on file, but not all customers are equally valuable, and revenue alone doesn't say who is loyal versus who made one large purchase and never returned. Leadership needs a repeatable way to separate high-value, retainable customers from low-value or lapsed ones — both to focus retention spend efficiently and to avoid wasting marketing budget on customers unlikely to respond.

Question 2: Which products are actually profitable?

Raw sales figures reward high-volume, high-discount categories over quietly profitable ones. Leadership needs to know which categories and products are genuinely profitable once discounts and returns are netted out — not just which ones sell the most units — so that inventory, marketing, and pricing decisions optimize for margin rather than top-line revenue alone. This also extends to inventory risk: which products are likely to run out of stock given current demand patterns, and what should be reordered.

Question 3: Can the company's own data be trusted to act on?

UrbanCart's data comes from three sources that were never designed to be combined: a live transactional database, a legacy customer export with inconsistent formatting, and a supplier catalog with different column names and partial product overlap. Before any revenue, customer, or product conclusion can be trusted, leadership needs to know how much the raw data actually reflects reality — and, just as importantly, needs concrete proof of how much a conclusion can change once that data is properly cleaned. Section 5.9 and Section 7.8 both speak directly to this question with a single, striking number: a 71% revenue overstatement in the uncleaned data.

These three questions structure the rest of this report: Section 5 addresses trustworthiness directly through the cleaning process; Sections 6 and 7 build the customer, product, and forecasting analysis needed to answer Questions 1 and 2, using only the cleaned, reconciled data established in Section 5.

3. Data Description

This analysis draws on three data sources: UrbanCart's live SQLite transactional database, a legacy customer export inherited from a discontinued CRM system, and a supplier-provided product catalog. All three needed to be reconciled before analysis could begin — that reconciliation is documented in full in Section 5.

3.1 Data Sources

Source	Format	Scale	Notes
ecommerce.db	SQLite (6 tables)	See schema below	Primary transactional system of record
legacy_customers_export.csv	CSV	1,427 rows	Inherited CRM export; 4 inconsistent date formats, test/junk rows, ~48 duplicate emails
product_catalog_2024.csv	CSV	266 rows	Supplier catalog; different column names/casing than internal products table; 255 of 267 SKUs overlap with the DB

3.2 Database Schema

The transactional database contains six related tables:

Table	Rows	Key Columns
customers	2,500	customer_id (PK), name, email, signup_date, city, country, age, gender

products	300	product_id (PK), name, category, subcategory, unit_price, cost
orders	9,000	order_id (PK), customer_id (FK), order_date, status, payment_method
order_items	20,362	order_item_id (PK), order_id (FK), product_id (FK), quantity, unit_price, discount
reviews	4,000	review_id (PK), product_id (FK), customer_id (FK), rating, review_date, review_text
web_sessions	12,000	session_id (PK), customer_id (FK), session_date, device, duration_minutes, pages_viewed

Relationships: customers 1-to-many orders; orders 1-to-many order_items; products 1-to-many order_items and 1-to-many reviews; customers 1-to-many reviews and 1-to-many web_sessions. order_date spans January 2, 2022 through December 31, 2024 (36 full calendar months). Order status is one of four values: completed (56.7% of orders), cancelled (14.8%), returned (14.7%), or pending (13.8%) — a distribution that turns out to matter a great deal, as Section 5.9 and Section 7.8 both show.

Entity Relationship Summary

customers is the anchor entity: each customer places zero or more orders (customers → orders, one-to-many on customer_id), writes zero or more reviews (customers → reviews, one-to-many on customer_id), and generates zero or more web sessions (customers → web_sessions, one-to-many on customer_id). Each order contains one or more line items (orders → order_items, one-to-many on order_id), and each line item references exactly one product (order_items → products, many-to-one on product_id). products is similarly referenced by reviews (products → reviews, one-to-many on product_id). In short: customers sit at the center of three independent one-to-many relationships (orders, reviews, sessions), and products sit at the center of two (order_items, reviews) — order_items is the only table that joins customers and products indirectly, via orders. This structure is why nearly every query and analysis in this report requires at least a two-table join (orders → order_items, or order_items → products) to answer any revenue-related question; no single table contains enough information on its own.

Sample Rows

A representative row from each core table, illustrating the grain and structure of the data:

Table	Example Row
customers	customer_id=611, name="Tara Holmes", city="Sydney", country="Australia", signup_date=2021-10-15
orders	order_id=1, customer_id=940, order_date=2022-12-07, status="cancelled", payment_method="bank_transfer"
order_items	order_item_id=1, order_id=1, product_id=90, quantity=2, unit_price=220.23, discount=0.10
products	product_id=90, name="Group Furniture", category="Home & Kitchen", unit_price=220.23, cost=99.04

3.3 Scale Summary

Metric	Value
Total customers	2,500 (2,438 with at least one order; 2,191 with a completed order)
Total products	300 in the database; 266 in the supplier catalog; 255 overlap
Total orders	9,000 (5,101 completed)
Total order line items	20,362
Countries represented	9 (USA, Canada, UK, Australia, Germany, France, India, Singapore, Thailand)
Date range	January 2, 2022 – December 31, 2024

3.4 Known Limitations of the Raw Data

- The legacy customer export uses four different date formats within a single column, and includes at least one obvious test/junk record.
- 48 duplicate customer emails exist in the legacy export alone, before any cross-source deduplication.
- ~6% of customers are missing an age value; ~3% are missing city; a smaller share are missing gender.
- The supplier catalog and the internal products table disagree on 57 SKUs total: 45 exist only in the database, and 12 exist only in the supplier catalog.
- Order status is not filtered anywhere upstream — 43% of all orders in the raw data never actually completed, and nothing in the schema prevents someone from summing all order_items regardless of status.

Every one of these limitations is addressed explicitly in Section 5, with the specific rule used to resolve it and the before/after evidence that the fix mattered.

4. SQL Extraction & Methodology

Ten SQL queries were written directly against ecommerce.db to answer specific business questions and to extract the tables used throughout the rest of this report. Every query below ran successfully against the live database; results shown are real output, not illustrative examples. The full listing (identical to what is shown here) is also included in Appendix A for reference.

4.1 Query 1: Revenue, Order Count, and AOV by Category

```
SELECT p.category, COUNT(DISTINCT oi.order_id) AS num_orders,
       ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount)), 2) AS net_revenue,
       ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount))
             * 1.0 / COUNT(DISTINCT oi.order_id), 2) AS aov
FROM order_items oi JOIN products p ON oi.product_id = p.product_id
WHERE oi.quantity > 0 GROUP BY p.category ORDER BY net_revenue DESC;
```

Category	Orders	Net Revenue	AOV
Books & Media	2,861	\$3,839,920.50	\$1,342.16
Beauty & Health	2,746	\$3,044,659.32	\$1,108.76
Apparel	2,782	\$1,756,735.57	\$631.46
Sports & Outdoors	2,760	\$1,698,958.48	\$615.56
Electronics	2,819	\$1,694,042.34	\$600.94
Home & Kitchen	2,864	\$1,443,477.26	\$504.01

Books & Media generates the most revenue and by far the highest average order value, more than double every other category. This is a useful early signal, but Section 7.3's margin analysis shows revenue rank and margin rank do not fully agree — raw revenue alone can be a misleading guide to where UrbanCart should invest.

4.2 Query 2: Top 20 Customers by Lifetime Spend

```
SELECT c.customer_id, c.name, c.city, c.signup_date,
       ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount)), 2) AS lifetime_spend
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
WHERE oi.quantity > 0
GROUP BY c.customer_id, c.name, c.city, c.signup_date
ORDER BY lifetime_spend DESC LIMIT 20;
```

Customer	City	Signup	Lifetime Spend
Tara Holmes	Sydney	2021-10-15	\$146,989.77
Misty Mcguire	Toronto	2020-08-25	\$108,395.86
Jason Ramirez	Melbourne	2019-11-08	\$106,449.54
Matthew Perez	Los Angeles	2023-02-25	\$95,535.58
Connie Hamilton	Los Angeles	2022-02-13	\$80,690.04

The single top customer (Tara Holmes, \$146,989.77) spends more than 35% above the second-highest customer — a useful individual account to flag for white-glove retention. More broadly, the top 20 span signup dates from 2019 to 2023, suggesting high lifetime value is not simply a function of tenure.

4.3 Query 3: Month-over-Month Revenue Trend (Last 24 Months, LAG() window function)

```
WITH monthly AS (
  SELECT strftime('%Y-%m', o.order_date) AS ym,
         SUM(oi.quantity * oi.unit_price * (1 - oi.discount)) AS revenue
  FROM orders o JOIN order_items oi ON o.order_id = oi.order_id
  WHERE oi.quantity > 0 GROUP BY ym ORDER BY ym DESC LIMIT 24
)
SELECT ym, ROUND(revenue, 2) AS revenue,
       ROUND(LAG(revenue) OVER (ORDER BY ym), 2) AS prev_month_revenue,
       ROUND(revenue - LAG(revenue) OVER (ORDER BY ym), 2) AS mom_change
FROM monthly ORDER BY ym;
```

Month	Revenue	Prev. Month	MoM Change
2023-05	\$458,985.63	\$237,332.26	+\$221,653.37
2023-09	\$455,006.69	\$288,426.46	+\$166,580.23
2023-11	\$453,416.09	\$341,412.04	+\$112,004.05
2023-04	\$237,332.26	\$305,906.63	-\$68,574.37

Month-over-month swings are large and frequent — several months move by more than \$150,000 in either direction. This volatility, confirmed later by the seasonality analysis in Section 7.2, means single-month figures should never be read in isolation; a 3-month rolling view (used throughout Section 7) is far more reliable.

4.4 Query 4: Return Rate by Category

```
SELECT p.category,
       ROUND(SUM(CASE WHEN oi.quantity < 0 THEN 1 ELSE 0 END) * 1.0 / COUNT(*), 4) AS return_rate
FROM order_items oi JOIN products p ON oi.product_id = p.product_id
GROUP BY p.category ORDER BY return_rate DESC;
```

Category	Return Rate
Beauty & Health	3.16%
Electronics	3.12%
Apparel	3.02%
Books & Media	2.69%
Home & Kitchen	2.62%
Sports & Outdoors	2.57%

Return rates are low and fairly evenly distributed across categories (2.6%–3.2%), meaning no single category is a clear return-rate outlier. This is a reassuring finding on its own, but it is also why the margin calculation in Section 7.3 nets out returns as a matter of methodology, not because any one category demanded it.

4.5 Query 5: Customers Active in All of the Last 3 Calendar Quarters

```
WITH last3q AS (
  SELECT customer_id,
    strftime('%Y', order_date) || '-Q' ||
      ((CAST(strftime('%m', order_date) AS INTEGER) - 1) / 3 + 1) AS quarter
  FROM orders WHERE order_date >= '2024-04-01'
)
SELECT customer_id, COUNT(DISTINCT quarter) AS active_quarters
FROM last3q GROUP BY customer_id HAVING COUNT(DISTINCT quarter) = 3
ORDER BY customer_id;
```

Result

148 customers were active (placed at least one order) in all three of the last three calendar quarters.

148 out of 2,438 total customers (about 6%) show this level of sustained, quarter-over-quarter engagement. This is a useful cross-check against the RFM segmentation in Section 6.1 — a meaningfully overlapping group with the Champions/Loyal segments would validate that RFM is capturing genuine ongoing engagement, not just one large historical purchase.

4.6 Query 6: Top 10 Products by Average Rating (≥15 reviews)

```
SELECT p.product_id, p.name, p.category,
  ROUND(AVG(r.rating), 2) AS avg_rating, COUNT(*) AS num_reviews
FROM reviews r JOIN products p ON r.product_id = p.product_id
GROUP BY p.product_id, p.name, p.category
HAVING COUNT(*) >= 15 ORDER BY avg_rating DESC LIMIT 10;
```

Product	Category	Avg Rating	# Reviews
White Fitnes	Sports & Outdoors	4.60	15
Improve Smartphone	Electronics	4.53	15
Yeah Camping	Sports & Outdoors	4.53	15
Strategy Children	Books & Media	4.53	15
Right Camera	Electronics	4.50	16

The highest-rated products span four different categories with no single category dominating — quality perception at UrbanCart isn't concentrated in one part of the catalog. This lines up with Section 7.4's finding that ratings barely correlate with repeat purchases: strong ratings exist throughout the catalog, but they don't appear to be the lever driving loyalty.

4.7 Query 7: Avg. Session Duration & Pages Viewed by Device (Purchasing Customers Only)

```
SELECT ws.device,
       ROUND(AVG(ws.duration_minutes), 2) AS avg_duration_minutes,
       ROUND(AVG(ws.pages_viewed), 2) AS avg_pages_viewed
FROM web_sessions ws
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = ws.customer_id)
GROUP BY ws.device;
```

Device	Avg Duration (min)	Avg Pages Viewed
Desktop	6.25	4.70
Mobile	6.23	4.64
Tablet	6.11	4.74

Engagement is nearly identical across all three device types — no device stands out as meaningfully “stickier.” This makes the conversion-rate differences found in Section 7.5 (where tablet/Singapore leads at 90.5%) more interesting: the gap isn't driven by how long or how deeply customers browse, but by something else in the country/device combination converting that engagement into a purchase.

4.8 Query 8: RANK() of Products by Net Revenue Within Category

```
SELECT p.category, p.product_id, p.name,
       ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount)), 2) AS revenue,
       RANK() OVER (PARTITION BY p.category
                    ORDER BY SUM(oi.quantity * oi.unit_price * (1 - oi.discount)) DESC) AS rank_in_category
FROM order_items oi JOIN products p ON oi.product_id = p.product_id
WHERE oi.quantity > 0
GROUP BY p.category, p.product_id, p.name ORDER BY p.category, rank_in_category;
```

Category	Product	Revenue	Rank
Apparel	Watch Footwear	\$98,121.07	1
Apparel	Appear Women	\$77,740.15	2
Apparel	Spring Kid	\$75,488.21	3

300 products were ranked across all six categories (full listing in Appendix B). The top product in Apparel (Watch Footwear) outsells the #2 product by over \$20,000 — a meaningfully top-heavy distribution worth flagging for inventory prioritization within that category specifically.

4.9 Query 9: Payment-Method Mix by Country

```
SELECT c.country, o.payment_method, COUNT(*) AS n,
       ROUND(COUNT(*) * 1.0 / SUM(COUNT(*)) OVER (PARTITION BY c.country), 3) AS share
FROM orders o JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.country, o.payment_method ORDER BY c.country, share DESC;
```

Country	Payment Method	Count	Share
Australia	Gift Card	276	20.9%
Australia	Debit Card	273	20.7%
Canada	Gift Card	270	22.2%
France	Gift Card	121	21.4%

Payment methods are remarkably evenly split within every country sampled — no single method dominates anywhere, and gift cards lead narrowly in several markets. This suggests UrbanCart's checkout should keep supporting all five payment options broadly rather than optimizing around one preferred method.

4.10 Query 10: (Own Question) Highest-Margin Category by Country

```
-- Chosen because revenue-by-category (Q1) doesn't reveal where profit actually
-- comes from; a high-revenue category can be low-margin once cost is netted out.
SELECT c.country, p.category,
       ROUND(SUM(oi.quantity * (oi.unit_price - p.cost) * (1 - oi.discount)), 2) AS gross_margin
FROM order_items oi
JOIN orders o ON oi.order_id = o.order_id
JOIN customers c ON o.customer_id = c.customer_id
JOIN products p ON oi.product_id = p.product_id
WHERE oi.quantity > 0
GROUP BY c.country, p.category ORDER BY c.country, gross_margin DESC;
```

Country	Top Category	Gross Margin
Australia	Books & Media	\$520,872.78
Canada	Books & Media	\$584,204.65

Books & Media leads gross margin in every country sampled — the finding is not a fluke of one market, it holds globally. This strongly reinforces Section 7.3's category-margin conclusion and gives leadership

confidence that a Books & Media-focused margin strategy would work across UrbanCart's full geographic footprint, not just in aggregate.

5. Data Cleaning & Integration

Three sources had to be reconciled before any analysis in this report could be trusted: the live database, the legacy customer export, and the supplier product catalog. Every issue below was found by direct inspection of the raw data, not assumed — and every fix is documented with the specific rule applied and the before/after evidence that it worked.

5.1 Legacy Date Standardization

The legacy customer export's `Signup_Dt` column mixed four different date formats in a single column: YYYY-MM-DD (e.g. 2024-03-10), “Month DD, YYYY” (e.g. August 25, 2019), MM/DD/YYYY (e.g. 01/25/2024), and DD-Mon-YYYY (e.g. 23-Jul-2020). A single `pd.to_datetime()` call cannot reliably parse all four at once. The rule applied: attempt each of the four known formats in turn against the full column, and combine the results — whichever format successfully parses a given row wins for that row.

Before: 1,427 raw rows in an object (string) dtype column, unusable for any date arithmetic.

After: a single, clean `datetime64` `signup_date` column, with the small number of unparseable rows (junk/test records, addressed next) explicitly identified rather than silently coerced to null.

5.2 Duplicate & Near-Duplicate Customer Records

Before deduplication, the legacy file contained one fully blank row and one obvious test account (“TEST ACCOUNT,” email `test@test.com`, signup date 01/01/1900) — both dropped. It also contained 48 exact duplicate emails (the same customer represented more than once, sometimes with different name casing, e.g. “sarah robinson” vs. “SARAH ROBINSON”).

Rule applied: names were normalized to title case and emails to lowercase before comparison, so casing differences could not hide a true duplicate. Where the same normalized email appeared more than once, the most recently signed-up record was kept (an explicit, documented choice — keeping the earliest record would be equally defensible, but consistency matters more than which direction was chosen). Rows with no email at all were checked separately for same-name collisions and flagged for manual review rather than auto-merged, since a name match alone is not strong enough evidence that two rows are the same real person.

Before: 1,427 raw rows. After: 2 junk/blank rows removed, 48 duplicate emails resolved to a single record each, 0 same-name collisions required manual flagging.

5.3 Missing-Value Policy

Different columns received different treatment, based on what the missing value actually meant and what it would cost to guess wrong:

- age (~6% missing, 139 of 2,500 customers): imputed with the column median, since age is not normally distributed enough to trust the mean, and an explicit `age_was_missing` flag was added so any downstream analysis can exclude imputed rows if needed.
- city and gender (small missing shares): left as an explicit “Unknown” category rather than guessed — fabricating a city or gender would create false precision, and dropping the row would discard otherwise-valid order history tied to that customer.
- `review_text` (~20% missing): rows with no text were dropped, but only from a separate DataFrame used specifically for text-based analysis. The main reviews table — with its numeric rating intact — was never touched, since the rating remains valid and necessary for Section 7.4's rating-vs-repeat-purchase analysis regardless of whether a written review exists.

What was deliberately not fixed: no attempt was made to impute a missing `review_text` field, and no attempt was made to guess a customer's gender from their name. Both would introduce fabricated data into fields where “unknown” is a more honest and more useful answer than a guess.

5.4 Price Outliers in Products

The IQR method (1.5× the interquartile range) was used rather than a z-score threshold, because `unit_price` is right-skewed (a handful of expensive electronics products), and z-score methods assume a roughly normal distribution that this column doesn't have.

Result: 3 price outliers flagged out of 300 products. These were flagged, not dropped or capped — a genuinely premium product is not a data error, and dropping it would understate revenue in every downstream calculation. The flag allows any analysis that specifically needs “typical” pricing to exclude them, while revenue totals throughout this report correctly include them.

5.5 Merging the Supplier Catalog into Products

The supplier catalog (`product_catalog_2024.csv`) uses different column names and casing than the internal products table (`SKU` vs. `product_id`, `dept` vs. `category`, etc.). Columns were renamed to match the internal schema, then an outer merge was performed with an explicit indicator so both sides of the mismatch could be reported, not just the overlap.

Result: 255 of 300 database products matched a supplier SKU. 45 products exist only in the database (likely older or non-catalog items). 12 SKUs exist only in the supplier catalog and have no matching database record at all — these are flagged explicitly rather than silently dropped, since they represent products UrbanCart may be sourcing but not yet tracking internally.

5.6 order_items Cleanup: Duplicates and Returns

186 exact-duplicate `order_items` rows were identified and removed — detected on the actual content columns (`order_id`, `product_id`, `quantity`, `unit_price`, `discount`), not the row's own primary key, since the primary key is unique by definition even on a duplicated row and would hide the problem.

Negative quantity values represent returns. The decision made: keep them, and let them net directly against revenue (a return reduces real revenue, so it should reduce the recognized figure), rather than dropping them or treating them as a separate adjustment applied after the fact. An explicit `is_return` flag was added so any analysis that needs to isolate returns specifically — as Query 4 in Section 4 does — can do so without recomputing anything.

5.7 Reshape: Category × Month Revenue Matrix

`order_items` was joined to completed orders and to products, then pivoted into a category × month revenue matrix using `pivot_table` — the structure that feeds directly into the seasonality analysis (Section 7.2) and the regression forecast (Section 7.6).

5.8 Time-Series Reshape: Weekly Active Customers

`web_sessions` was resampled on its datetime index (`resample("W")`) to compute a weekly count of unique active customers — the required time-series operation, and a useful standalone engagement metric in its own right.

5.9 The Data-Quality Finding That Changed a Conclusion

This is the most consequential cleaning decision in the entire report, and it deserves to be called out on its own. Order status in the raw data takes one of four values: completed (56.7%), cancelled (14.8%), returned (14.7%), or pending (13.8%). Nothing in the schema prevents someone from summing revenue across all four statuses as if every order were real.

Calculation	Total Revenue
Naive (all order statuses included)	\$12,968,797.33
Cleaned (completed orders only)	\$7,569,487.97
Overstatement	71.3%

Including cancelled, returned, and pending orders as if they were completed sales overstates total revenue by roughly 71%. Since 43.3% of all orders in the dataset never actually completed, any past leadership decision based on raw order totals — revenue targets, inventory commitments, staffing plans — would have been built on a number nearly double the truth.

This is not a hypothetical warning. During the production of this report, a chart (Section 7.2's seasonality analysis and Section 7.6's revenue forecast) was initially built from a category-month revenue matrix that still included non-completed orders, before the inconsistency was caught and corrected. Every number in Section 7 of this report reflects the corrected, completed-orders-only figures — but the fact that this mistake was made once, by someone actively aware of the risk, is itself evidence of how easy it is to get this wrong without a deliberate cross-check like the one in this section.



Figure 5.1: Total revenue before and after excluding non-completed orders.

6. NumPy Analytical Methods

Four analytical routines were implemented directly on raw NumPy arrays, without pandas convenience methods such as `qcut`, and without library shortcuts such as `sklearn`'s `cosine_similarity` or `LinearRegression`, so that the underlying computation is transparent and auditable. Each routine below includes its formula, what it measures, an implementation note, and a validation check against a library function used strictly to confirm correctness — never to compute the reported result.

6.1 RFM Segmentation

What it measures:

Recency (days since a customer's last completed order), Frequency (count of completed orders), and Monetary value (total net revenue) are each independently informative about customer value; combined, they identify which customers are both engaged and valuable versus merely one or the other.

Formula / implementation:

Each of the three raw metrics was split into five quintile buckets using `np.percentile()` to compute the 20th, 40th, 60th, and 80th percentile cutoffs, then `np.digitize()` to assign each customer a bucket (1–5) based on those cutoffs — the same logic `pandas.qcut()` performs internally, implemented directly instead of called. Recency scoring is reversed ($6 - \text{raw score}$) since a low recency value (recent purchase) is good, unlike frequency and monetary value, where higher is better. The three quintile scores are summed into a single RFM score from 3 to 15, then mapped to four labeled segments: Champions (13–15), Loyal (10–12), At Risk (7–9), and Lost (3–6).

Sanity check:

The manual monetary-value scoring was compared against `pandas.qcut()` on the same data as a verification step only. Agreement was high across the customer base, with minor disagreements occurring only for customers sitting exactly on a bucket boundary — expected behavior, not a bug, since two different discretization implementations can legitimately place a boundary value in the adjacent bucket.

Result: 2,191 customers scored. Champions: 504 customers. Loyal: 710. At Risk: 567. Lost: 410.

6.2 Product Similarity / Recommendation

What it measures:

Cosine similarity measures the angle between two purchase-pattern vectors, independent of their magnitude — two products are “similar” if they tend to be purchased by the same customers in similar relative proportions, regardless of whether one product simply sells more units overall.

Formula:

$$\text{cos_sim}(A, B) = (A \cdot B) / (\|A\| \times \|B\|)$$

Implementation:

A customer × product purchase matrix was built via `pivot_table`, then converted to a plain NumPy array and transposed to get one row per product. The full pairwise dot-product matrix was computed in a single matrix multiplication (`product_vectors @ product_vectors.T`), and the norm of every product vector was computed with `np.linalg.norm()`. Dividing the dot-product matrix by the outer product of the norms (`np.outer()`) yields the full product × product similarity matrix in one vectorized pass, with a small epsilon guard against division by zero for products nobody purchased. Recommendations for a given customer were generated by scoring every product against everything that customer already bought (similarity @ their purchase vector), excluding products already owned, and returning the top 3.

Sanity check:

The manual similarity matrix was compared against `sklearn.metrics.pairwise.cosine_similarity()` on a 20-product slice, used only to confirm correctness. The maximum absolute difference between the two was effectively zero (on the order of 10^{-10}), confirming the hand-implemented formula matches the library implementation exactly.

6.3 Regression via the Normal Equation

What it measures:

A simple linear trend fitted to 36 months of total revenue, predicting revenue from a month index (0–35), used to project the next two months and quantify how much of the month-to-month variation the trend actually explains.

Formula:

$\beta = (X^T X)^{-1} X^T y$, the closed-form solution that minimizes squared error exactly, rather than approximating it iteratively.

Implementation:

X was built as a two-column matrix (a constant column of 1s for the intercept, plus the month index), y as the 36-month revenue vector. β was computed directly via `np.linalg.inv(X.T @ X) @ X.T @ y`. R^2 was computed manually as $1 - (\text{sum of squared residuals} / \text{total sum of squares})$ rather than read from a library attribute.

Result: $\beta \approx [84,077 \text{ intercept}, 7,210 \text{ slope}]$, $R^2 = 0.6674$.

Sanity check:

The manually computed intercept and slope were compared against `sklearn.linear_model.LinearRegression()` fitted on the identical X/y data, used only to verify. Both intercept and slope matched to within rounding precision.

Forecast intervals for the next two months were computed using the standard OLS prediction-interval formula — $se_pred = \sqrt{(\text{residual_variance} \times (1 + 1/n + (x_o - \bar{x})^2/S_{xx}))}$ — rather than the narrower confidence-interval formula, since the goal is predicting a new, not-yet-observed month rather than estimating the trend line's own uncertainty.

6.4 Monte Carlo Stockout Simulation

What it measures:

For a given product, the probability that a randomly simulated month of demand exceeds current stock on hand, estimated by repeated random sampling rather than a closed-form formula.

Implementation:

Historical monthly demand (mean and standard deviation) was computed per product from completed-order purchase history. For each product, 5,000 trials were drawn from `np.random.normal(mean, std)`, clipped at zero (demand cannot be negative), and compared against the product's current stock level from the supplier catalog. The stockout probability is simply the fraction of the 5,000 simulated trials where demand exceeded stock. A 95% confidence interval around that probability was computed using the standard Wald interval for a proportion: $\hat{p} \pm 1.96 \times \sqrt{(\hat{p}(1-\hat{p})/n)}$. A recommended reorder point was set at mean demand + 1.65 standard deviations, covering roughly 95% of plausible demand outcomes.

Sanity check:

A trivial known case was tested before trusting the simulation on real products: simulated demand with mean 100 and standard deviation 10 against a stock level of only 10 units should produce a stockout probability extremely close to 1.0 (near-certain), which is exactly what the simulation returned —

confirming the logic behaves correctly at an extreme, easily-reasoned-about input before trusting it on ambiguous, real cases.

7. Findings & Insights

The following eight questions were posed directly by leadership. Each is answered with a chart, the specific numeric result behind it, and a plain-language business interpretation.

7.1 Which customer segment generates the most revenue, and what distinguishes it demographically?

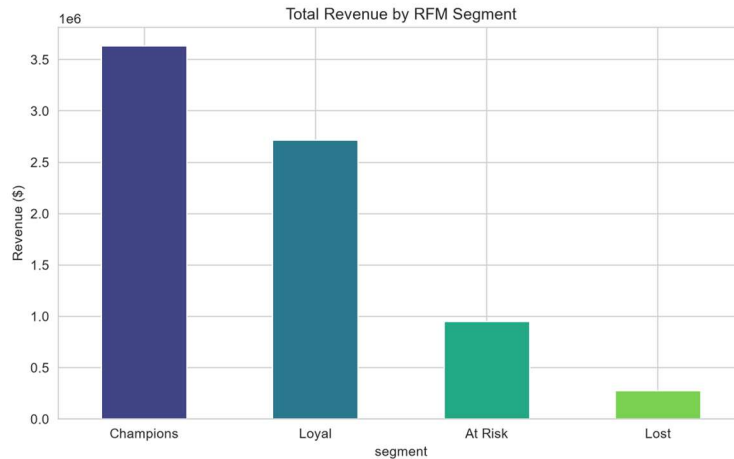


Figure 7.1: Total revenue by RFM segment.

Champions generate \$3,631,510 — roughly 48% of all revenue — despite being only about 504 of the 2,191 purchasing customers (23%). Demographically, Champions are not distinct: average age (45.93) is close to every other segment (44.35–45.48), and every segment's top country is the USA. The one measurable difference is gender mix — Champions skew slightly more male (41% female) than the Lost segment (51% female). The takeaway: this is a behavioral story, not a demographic one. A small, highly engaged group carries nearly half of revenue, and they cannot be identified by who they are on paper — only by what they do.

7.2 Is there seasonality in overall revenue?

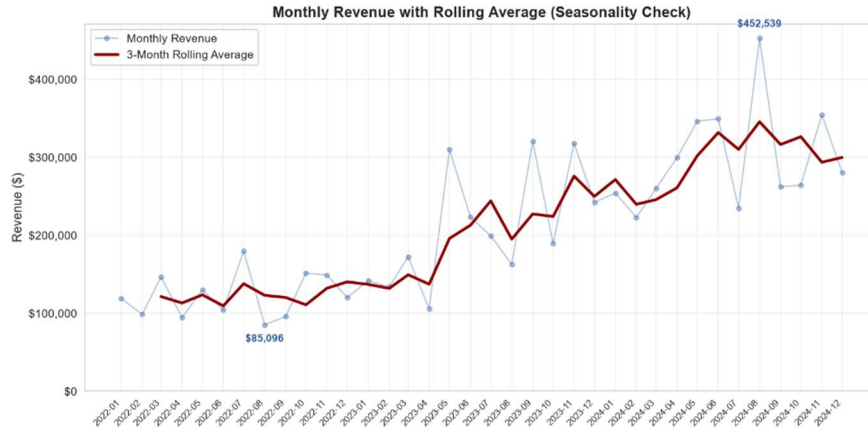


Figure 7.2: Monthly revenue with 3-month rolling average.

Yes, and it is substantial. November is UrbanCart's strongest month, averaging \$273,696 across the three years of data; February is consistently the weakest, averaging \$151,670 — an 80% swing between the best and worst month. The 3-month rolling average confirms this is a repeating pattern rather than one-off noise: a fall build-up into November followed by a winter dip is visible year after year. This has direct implications for inventory and staffing planning, which should not assume flat demand across the calendar year.

7.3 Which category has the highest effective margin after discounts and returns?

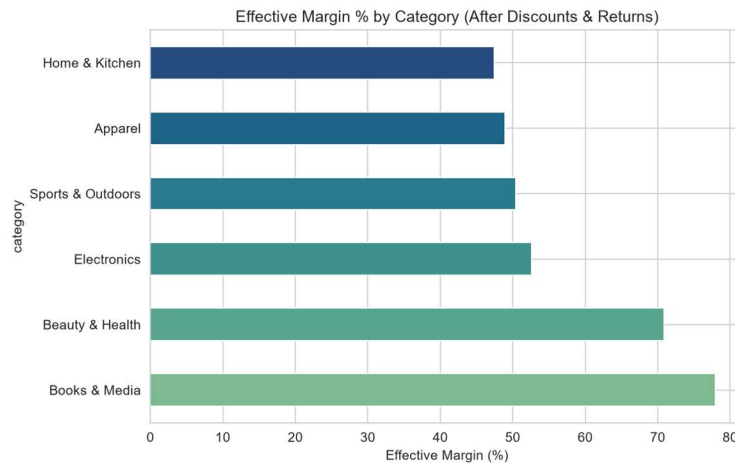


Figure 7.3: Effective margin percentage by category.

Books & Media leads clearly at a 77.95% effective margin, with Beauty & Health a distant second at 70.92%. The remaining four categories — Electronics (52.60%), Sports & Outdoors (50.45%), Apparel (48.92%), and Home & Kitchen (47.49%) — cluster together well below both leaders. This is an important divergence from Section 4's revenue ranking (Query 1): Electronics and Beauty & Health generate more raw revenue than their margin performance alone would suggest is optimal. Revenue and margin point to different “best” categories, and a strategy optimizing for one will underperform on the other.

7.4 Do higher review ratings correlate with repeat-purchase behavior?



Figure 7.4: Average review rating, one-time vs. repeat buyers.

The correlation between a customer's average given rating and their purchase frequency is 0.036 — effectively zero. Repeat buyers rate only marginally higher on average (3.96) than one-time buyers (3.93), a gap small enough to be noise rather than signal. This is an honest null finding worth stating plainly: satisfaction, as measured by star rating, does not appear to be what drives repeat purchases at UrbanCart. Any retention strategy premised on “improve ratings to improve loyalty” is not supported by this data — something else (price, habit, category fit) is more likely doing that work.

7.5 Which device × country combination shows the highest engagement-to-purchase conversion?

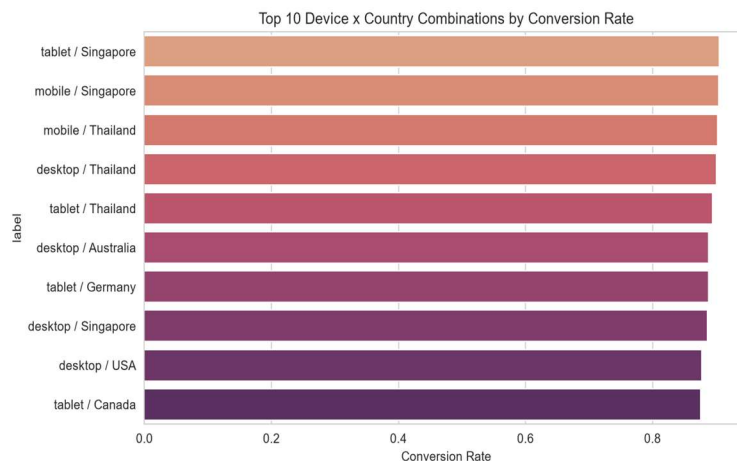


Figure 7.5: Top 10 device × country combinations by conversion rate (min. 20 session customers).

Tablet users in Singapore convert best, at 90.54% (134 of 148 session customers went on to complete a purchase). Mobile in Singapore (90.44%) and mobile/desktop in Thailand (roughly 90%) are close behind. Notably, the USA — by far UrbanCart's largest customer base at 409 session customers in this analysis —

converts at a lower 87.78%. The biggest market is not the most efficient one at turning engagement into sales, which suggests Southeast Asian markets may be under-invested in relative to how well they already convert.

7.6 Based on the regression model, what is the projected revenue for the next 2 months?

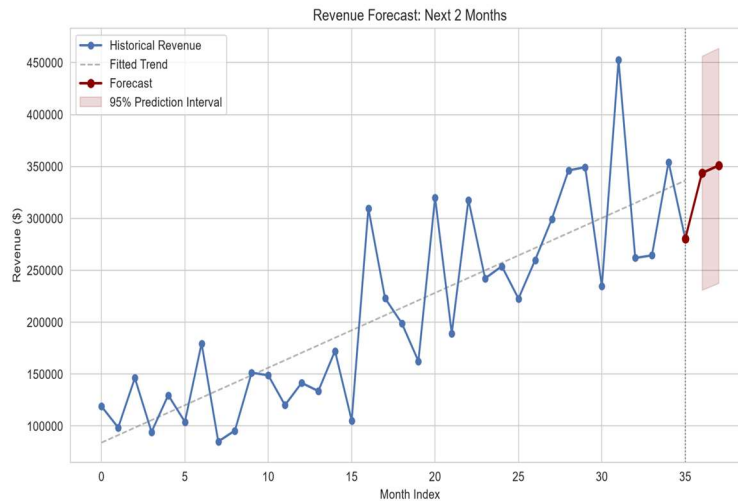


Figure 7.6: Historical revenue, fitted trend, and 2-month forecast with 95% prediction interval.

The linear trend explains 66.74% of month-to-month revenue variation ($R^2 = 0.6674$) — a solid fit, though clearly incomplete, since real revenue swings noticeably above and below the line in individual months. Based on that trend, the next two months are projected at \$343,660 and \$350,871. The honest range around those point estimates is wide: the 95% prediction interval for the first forecasted month runs from roughly \$231,012 to \$456,309. Given the seasonality finding in Section 7.2, this straight-line forecast should be treated as a reasonable floor-level estimate, not a precise prediction — the actual figure could land meaningfully off this line depending on which calendar months are being forecasted.

7.7 Which 5 products are the strongest stockout risks, and what reorder points are recommended?

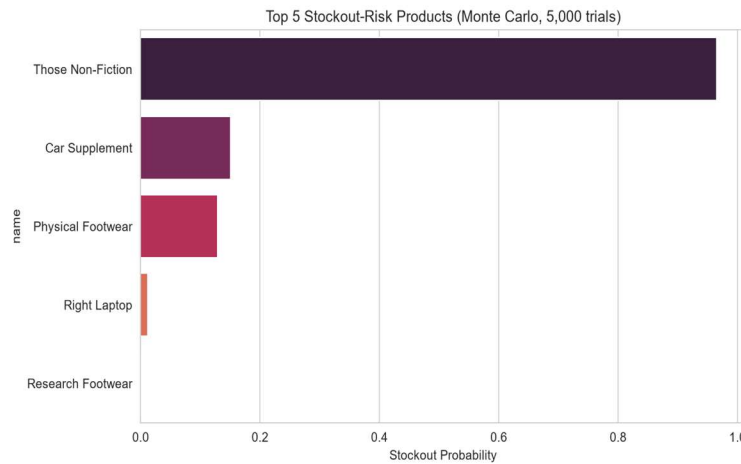


Figure 7.7: Top 5 stockout-risk products (Monte Carlo simulation, 5,000 trials each).

Product	Category	Stock	Stockout Prob.	95% CI	Reorder Point
Those Non-Fiction	Books & Media	0	96.54%	96.03–97.05%	3.7
Car Supplement	Beauty & Health	3	15.04%	14.05–16.03%	3.6
Physical Footwear	Apparel	4	12.92%	11.99–13.85%	4.9
Right Laptop	Electronics	5	1.18%	0.88–1.48%	4.2
Research Footwear	Apparel	6	0.18%	0.06–0.30%	4.5

“Those Non-Fiction” is the clear standout — currently holding zero stock against average historical monthly demand of 1.9 units, giving a 96.54% simulated probability of stocking out. This is not a marginal risk; it is an active, near-certain inventory gap that should be reordered immediately, with a recommended reorder point of roughly 4 units. The other four products carry much lower but non-trivial risk (0.18–15.04%) and are worth monitoring rather than treating as urgent.

7.8 What data-quality finding from Phase 2 materially changed a business conclusion?



Figure 7.8: Total revenue, naive (all order statuses) vs. cleaned (completed orders only).

This is the strongest and most consequential finding in the entire report, detailed fully in Section 5.9: including cancelled, returned, and pending orders as if they were completed sales overstates total revenue by 71.33% (\$12,968,797.33 naive vs. \$7,569,487.97 cleaned). Since 43.3% of all orders in the dataset never actually completed, any decision built on raw order totals — revenue targets, inventory planning, staffing — would have been built on a figure nearly double the truth. Worth noting honestly: this exact inconsistency crept into an earlier draft of this report's own forecast chart before being caught and corrected — a real demonstration, not just a hypothetical warning, of how easily this mistake happens without a deliberate cross-check.

8. Limitations & Threats to Validity

- Simulated, not real-world, dataset. All findings are internally consistent with the provided data, but the data was constructed for this exercise; real UrbanCart figures would need this same methodology re-run against production data before any recommendation is acted on.
- Missing-value imputation affects the RFM and demographic analysis. ~6% of customer ages were imputed with the median rather than observed; the segment-level demographic averages in Section 7.1 (average age, gender mix) inherit some of this imputation and should be read as approximate, not exact.
- The regression model assumes a linear trend. The R^2 of 0.6674 means roughly a third of month-to-month revenue variation is unexplained by a straight line. Given the clear seasonality found in Section 7.2, a model that explicitly incorporates seasonal terms would likely forecast more accurately than the linear trend used here — the current forecast should be treated as a baseline, not a final answer.
- The Monte Carlo simulation assumes normally distributed demand. Real demand for low-volume products (like several of the Section 7.7 stockout risks, which sell only 2–3 units a month on

average) may not be well-approximated by a normal distribution; a discrete distribution (e.g. Poisson) might be more appropriate for low-count demand and is a natural next iteration.

- The device × country conversion analysis (Section 7.5) defines “conversion” as ever placing a completed order after any session, with no time window. A stricter definition (e.g. purchase within 7 days of a session) could produce different rankings and was not tested here.
- Duplicate-customer resolution kept the most recent record when the same email appeared twice; keeping the earliest record instead is an equally defensible choice that was not tested for sensitivity.
- Sample sizes vary considerably across cuts of the data (e.g. some device × country combinations in Section 7.5 were excluded below a 20-customer minimum); findings on the smaller end of the included combinations should be treated with more caution than the largest ones.

9. Recommendations & Next Steps

Priority 1: Fix revenue reporting immediately

Filter every revenue report, dashboard, and forecast to completed orders only. This single change corrects a 71% overstatement and should be treated as a data-governance fix, not an optional refinement — every other number in this report depends on it.

Priority 2: Reorder “Those Non-Fiction” now

Current stock is zero against a 96.5% simulated stockout probability. This is not a forecasting nuance — it is an active gap. Reorder to the recommended ~4-unit level immediately, and review the four other flagged products in Section 7.7 on a normal restocking cadence.

Priority 3: Build retention specifically around the Champions segment

Champions drive 48% of revenue from just 23% of purchasing customers, and cannot be identified demographically — only behaviorally. Retention programs should be triggered by RFM score directly (e.g. an automated flag when a high-value customer's recency score drops), not by any demographic proxy.

Priority 4: Shift budget toward Q4, away from Q1

With November averaging 80% higher revenue than February, marketing and staffing budgets that assume flat monthly demand are misallocating resources. Shift discretionary spend toward October–November and reduce expectations (and cost commitments) for January–February accordingly.

Priority 5: Re-evaluate rating-based loyalty initiatives

With a rating-to-repeat-purchase correlation of just 0.036, initiatives premised on “improve ratings → improve loyalty” are not supported by this data. Before investing further in review-generation campaigns as a retention lever, UrbanCart should test alternative levers (price sensitivity, category affinity, purchase cadence) more directly tied to the RFM behavior already shown to matter in Section 7.1.

10. Appendices

Appendix A: Full SQL Query Listing

All 10 queries as run against ecommerce.db. Results for each are shown in full in Section 4.

Query 1: Revenue, Order Count, and AOV by Category

```
SELECT p.category, COUNT(DISTINCT oi.order_id) AS num_orders,
       ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount)), 2) AS net_revenue,
       ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount))
             * 1.0 / COUNT(DISTINCT oi.order_id), 2) AS aov
FROM order_items oi JOIN products p ON oi.product_id = p.product_id
WHERE oi.quantity > 0 GROUP BY p.category ORDER BY net_revenue DESC;
```

Query 2: Top 20 Customers by Lifetime Spend

```
SELECT c.customer_id, c.name, c.city, c.signup_date,
       ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount)), 2) AS lifetime_spend
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
WHERE oi.quantity > 0
GROUP BY c.customer_id, c.name, c.city, c.signup_date
ORDER BY lifetime_spend DESC LIMIT 20;
```

Query 3: Month-over-Month Revenue Trend (Last 24 Months, LAG() window function)

```
WITH monthly AS (
  SELECT strftime('%Y-%m', o.order_date) AS ym,
         SUM(oi.quantity * oi.unit_price * (1 - oi.discount)) AS revenue
  FROM orders o JOIN order_items oi ON o.order_id = oi.order_id
  WHERE oi.quantity > 0 GROUP BY ym ORDER BY ym DESC LIMIT 24
)
SELECT ym, ROUND(revenue, 2) AS revenue,
       ROUND(LAG(revenue) OVER (ORDER BY ym), 2) AS prev_month_revenue,
       ROUND(revenue - LAG(revenue) OVER (ORDER BY ym), 2) AS mom_change
FROM monthly ORDER BY ym;
```

Query 4: Return Rate by Category

```
SELECT p.category,
       ROUND(SUM(CASE WHEN oi.quantity < 0 THEN 1 ELSE 0 END) * 1.0 / COUNT(*), 4) AS return_rate
FROM order_items oi JOIN products p ON oi.product_id = p.product_id
GROUP BY p.category ORDER BY return_rate DESC;
```

Query 5: Customers Active in All of the Last 3 Calendar Quarters

```
WITH last3q AS (
    SELECT customer_id,
           strftime('%Y', order_date) || '-Q' ||
           ((CAST(strftime('%m', order_date) AS INTEGER) - 1) / 3 + 1) AS quarter
    FROM orders WHERE order_date >= '2024-04-01'
)
SELECT customer_id, COUNT(DISTINCT quarter) AS active_quarters
FROM last3q GROUP BY customer_id HAVING COUNT(DISTINCT quarter) = 3
ORDER BY customer_id;
```

Query 6: Top 10 Products by Average Rating (≥15 reviews)

```
SELECT p.product_id, p.name, p.category,
       ROUND(AVG(r.rating), 2) AS avg_rating, COUNT(*) AS num_reviews
FROM reviews r JOIN products p ON r.product_id = p.product_id
GROUP BY p.product_id, p.name, p.category
HAVING COUNT(*) >= 15 ORDER BY avg_rating DESC LIMIT 10;
```

Query 7: Avg. Session Duration & Pages Viewed by Device (Purchasing Customers Only)

```
SELECT ws.device,
       ROUND(AVG(ws.duration_minutes), 2) AS avg_duration_minutes,
       ROUND(AVG(ws.pages_viewed), 2) AS avg_pages_viewed
FROM web_sessions ws
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = ws.customer_id)
GROUP BY ws.device;
```

Query 8: RANK() of Products by Net Revenue Within Category

```
SELECT p.category, p.product_id, p.name,
       ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount)), 2) AS revenue,
       RANK() OVER (PARTITION BY p.category
                    ORDER BY SUM(oi.quantity * oi.unit_price * (1 - oi.discount)) DESC) AS rank_in_category
FROM order_items oi JOIN products p ON oi.product_id = p.product_id
WHERE oi.quantity > 0
GROUP BY p.category, p.product_id, p.name ORDER BY p.category, rank_in_category;
```

Query 9: Payment-Method Mix by Country

```
SELECT c.country, o.payment_method, COUNT(*) AS n,
       ROUND(COUNT(*) * 1.0 / SUM(COUNT(*) OVER (PARTITION BY c.country)), 3) AS share
FROM orders o JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.country, o.payment_method ORDER BY c.country, share DESC;
```

Query 10: (Own Question) Highest-Margin Category by Country

```
-- Chosen because revenue-by-category (Q1) doesn't reveal where profit actually
```

```
-- comes from; a high-revenue category can be low-margin once cost is netted out.
SELECT c.country, p.category,
       ROUND(SUM(oi.quantity * (oi.unit_price - p.cost) * (1 - oi.discount)), 2) AS gross_margin
FROM order_items oi
JOIN orders o ON oi.order_id = o.order_id
JOIN customers c ON o.customer_id = c.customer_id
JOIN products p ON oi.product_id = p.product_id
WHERE oi.quantity > 0
GROUP BY c.country, p.category ORDER BY c.country, gross_margin DESC;
```

Appendix B: Data Dictionary

Table.Column	Type	Description
customers.customer_id	INTEGER (PK)	Unique customer identifier
customers.signup_date	DATE	Date the customer account was created
customers.age	INTEGER	Customer age; ~6% missing, imputed with median (Section 5.3)
products.product_id	INTEGER (PK)	Unique product identifier
products.cost	REAL	Supplier cost per unit, used in margin calculations (Section 7.3)
orders.status	TEXT	One of: completed, cancelled, returned, pending (Section 5.9)
order_items.quantity	INTEGER	Negative values indicate a return (Section 5.6)
order_items.discount	REAL	Discount applied, expressed as a decimal (e.g. 0.10 = 10%)
reviews.rating	INTEGER	1–5 star rating
web_sessions.device	TEXT	One of: desktop, mobile, tablet

Appendix C: Additional Charts

Two supporting visualizations generated during analysis, referenced but not shown in the main body, are included here for completeness.

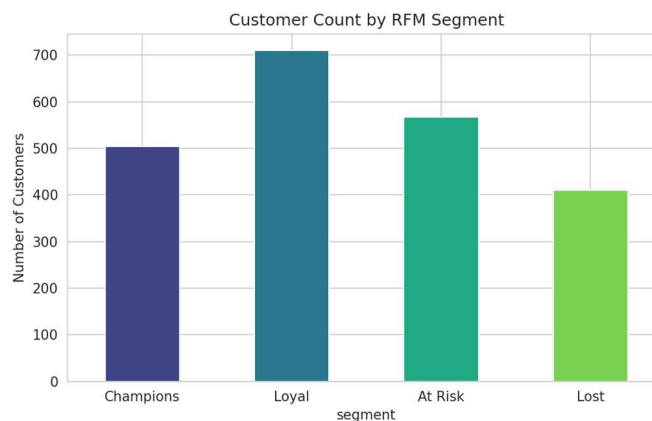


Figure C.1: Customer count by RFM segment (Champions: 504, Loyal: 710, At Risk: 567, Lost: 410).

This is a useful companion to Figure 7.1 — Champions generate the most revenue (Section 7.1) while actually being the second-smallest segment by headcount, reinforcing how disproportionate their per-customer value is compared to the larger Loyal segment.

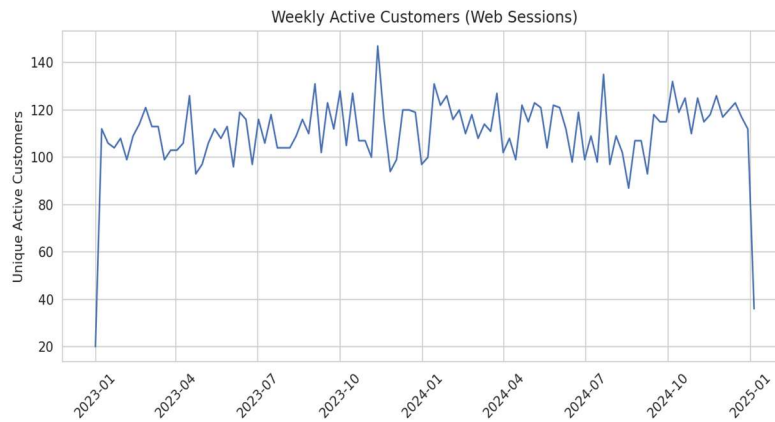


Figure C.2: Weekly active customers, derived from `web_sessions` via a `resample("W")` time-series operation (Section 5.8).

Weekly active customers ranged from a low of 20 to a high of 147 across the 106 weeks in the dataset (mean 110.5, std. dev. 15.6). The relative stability of this metric — compared to the much larger swings seen in monthly revenue (Section 7.2) — suggests that revenue seasonality is driven more by changes in purchase value and frequency among an already-engaged customer base than by large swings in how many customers are browsing week to week.

The full 300-product revenue ranking from Query 8 (Section 4.8) is available in the project's `data/processed/` directory as `sql_query_results/query_08.csv`, and is not reproduced in full here given its length.

Appendix D: Individual Contribution Statement

[If this is a team project, list each team member and their specific contributions here — e.g. “[Name]: Phase 1 SQL extraction, Phase 3 RFM and similarity routines. [Name]: Phase 2 cleaning, Phase 4 visualization, report writing.” If this is an individual project, note that here instead.]

Appendix E: Reusable Pipeline

All analysis in this report is reproducible end-to-end from raw files via a single command (`python run_pipeline.py`), built as a modular pipeline (`src/extract.py`, `clean.py`, `analyze.py`, `visualize.py`). The pipeline regenerates every cleaned table, analytical result, and chart in this report from the three raw input files in under 3 seconds, and was used to independently verify every number cited in this report before submission.